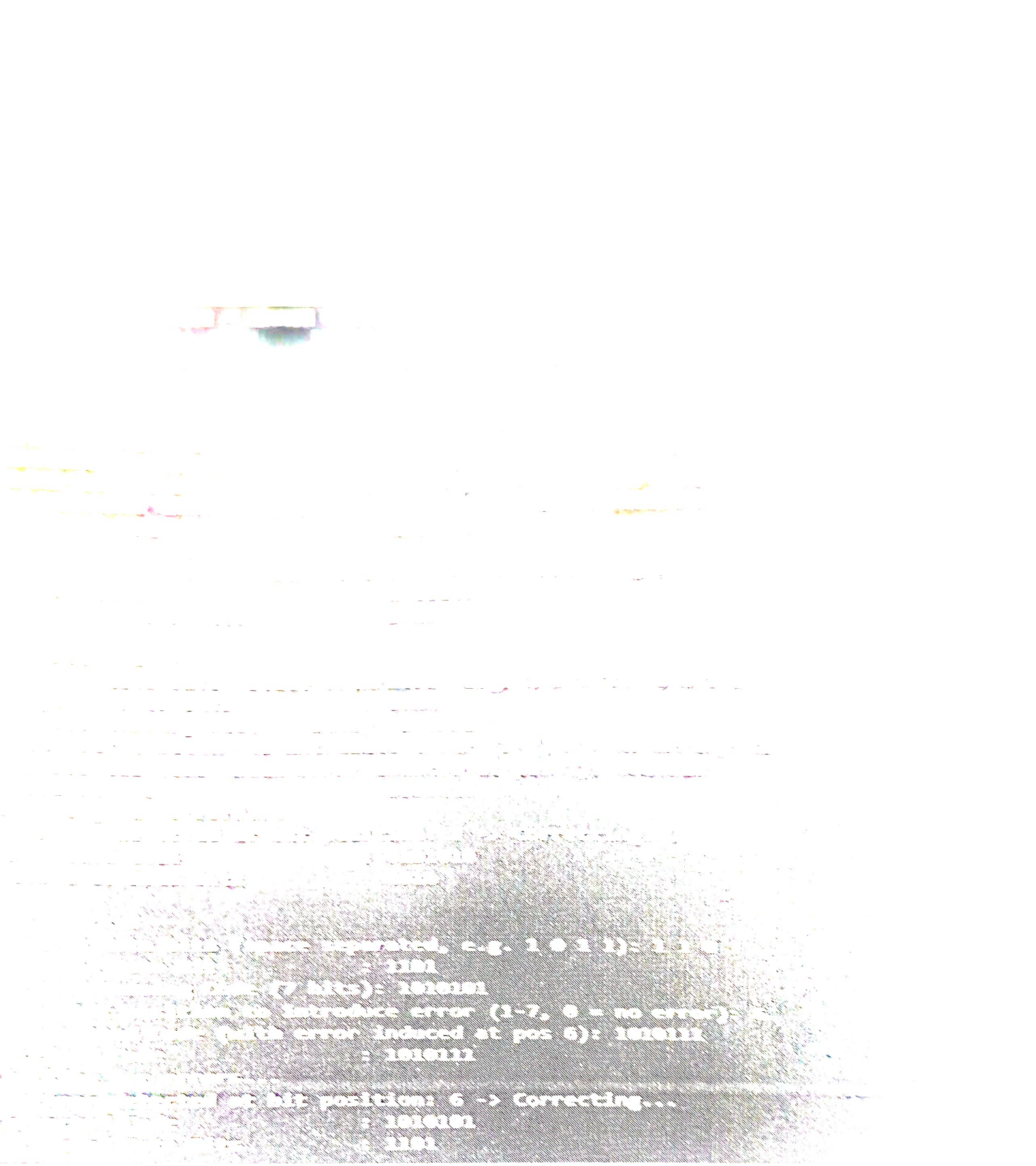
EXP: 6 ERROR DETECTION AND CORRECTION
13-08-2024 USING HAMMING CODE

Aim:
To write a program to implement error detection and correction using the hamming code.

ALGORITHM:

1. Start the program.
2. Read the number of 4 bit data blocks. (n).
   Reading the input data block D-d1 d2 d3 d4
   Perform hamming encoding
   a) Arrange 1, 2, and 4 ed and data bites at position 3, 5, 6 ed 7.
   b) Compute the ~~parity~~ parity bites at position
      $P1 = d1 \text{ XOR } d2 \text{ XOR } d4$ (Cover bit 1, 3, 5, 7)
      $P2 = d1 \text{ XOR } d3 \text{ XOR } d4$ (cover bit 2, 3, 6, 7)
      $P4 = d2 \text{ XOR } d3 \text{ XOR } d4$ (Cover bit 4, 5, 6, 7)
4. Simulate transmission by introducing a single bit error at a chosen bit position in the code word.
5. At the reciver recompute the check bits from the reciver code words
   $C1 = P1 \text{ XOR } d1 \text{ XOR } d2 \text{ XOR } d4$
   $C2 = P2 \text{ XOR } d1 \text{ XOR } d3 \text{ XOR } d4$
   $C3 = P4 \text{ XOR } d2 \text{ XOR } d3 \text{ XOR } d4.$


Enter 4 data bits (e.g. 1 0 1 1): 1 0 1 1
Encoded: 1011010
Introduce error (1-7, 0 = no error): 6
Received (6 error induced at pos 6): 1010110
Syndrome: 110
Error at bit position 6 -> Correcting...
Corrected: 1011010
Decoded data bits: 1011

6. Compute Error position = $C_4 C_2 C_1$

7. If error position = 0, no error has occurred.

8. Extract the orginal 4 data is inverted to Correct the error.

9. Repeat steps 3 to 8 for all n blocks in the data stream

10. Stop the program.

PROGRAM:

```
def encode(d)
""d: list of 4 bit [d1, d2, d3, d4] -> return.
7 bit hemming code list positions (1-indexed): 1=P1
2=P2 3=P3 4=P4 5=d2 6=d3 7=d4 """
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
h[0] = h[2] ^ h[4] ^ h[6]
h[1] = h[2] ^ h[5] ^ h[6]
h[3] = h[4] ^ h[5] ^ h[6]
```

```
def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]

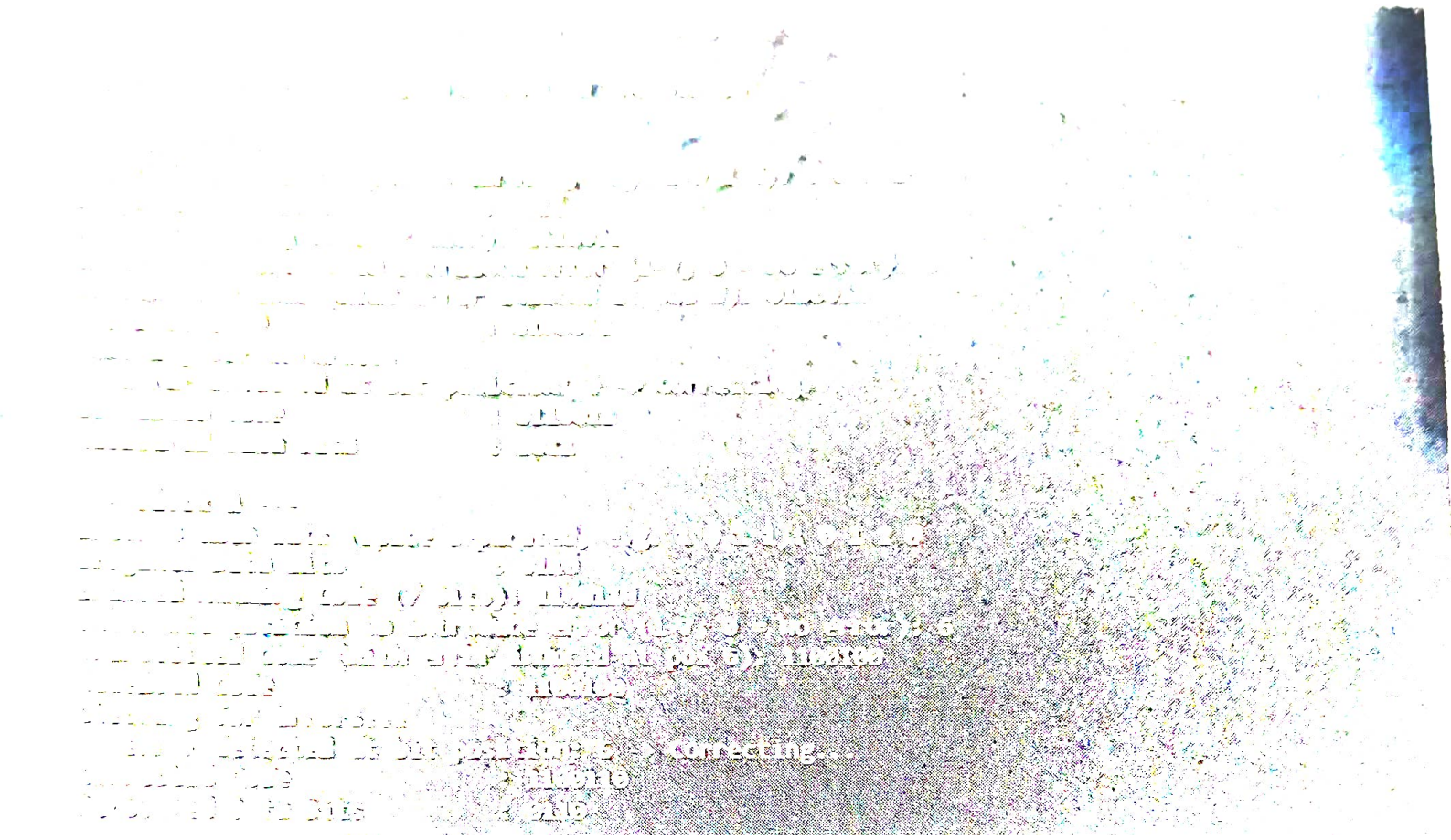
error_pos = c1*1 + c2*2 + c4*4

    if error_pos != 0
        print(f"Error detected at bit position {error_pos}.
                Correcting...")
        h[error_pos-1] ^= 1

    else:
        print("No error detected")
    return corr_pos.

def main()
    print("== HAMMING CODE - ERROR ===\n")
    n = int(input("Enter no of 4 bit data blocks:\n"))
    for b in range(1, n+1):
        print(f"\n--- Block {b} ---")
        h = encode(d)
        print(f"original data bits : {to_str(d)}")

if __name__ == "__main__"
    main()
```



Result:

The Program to implement error detection and correction using the hamming code concept was successfully written in Python, executed and verified using a sample input data stream.